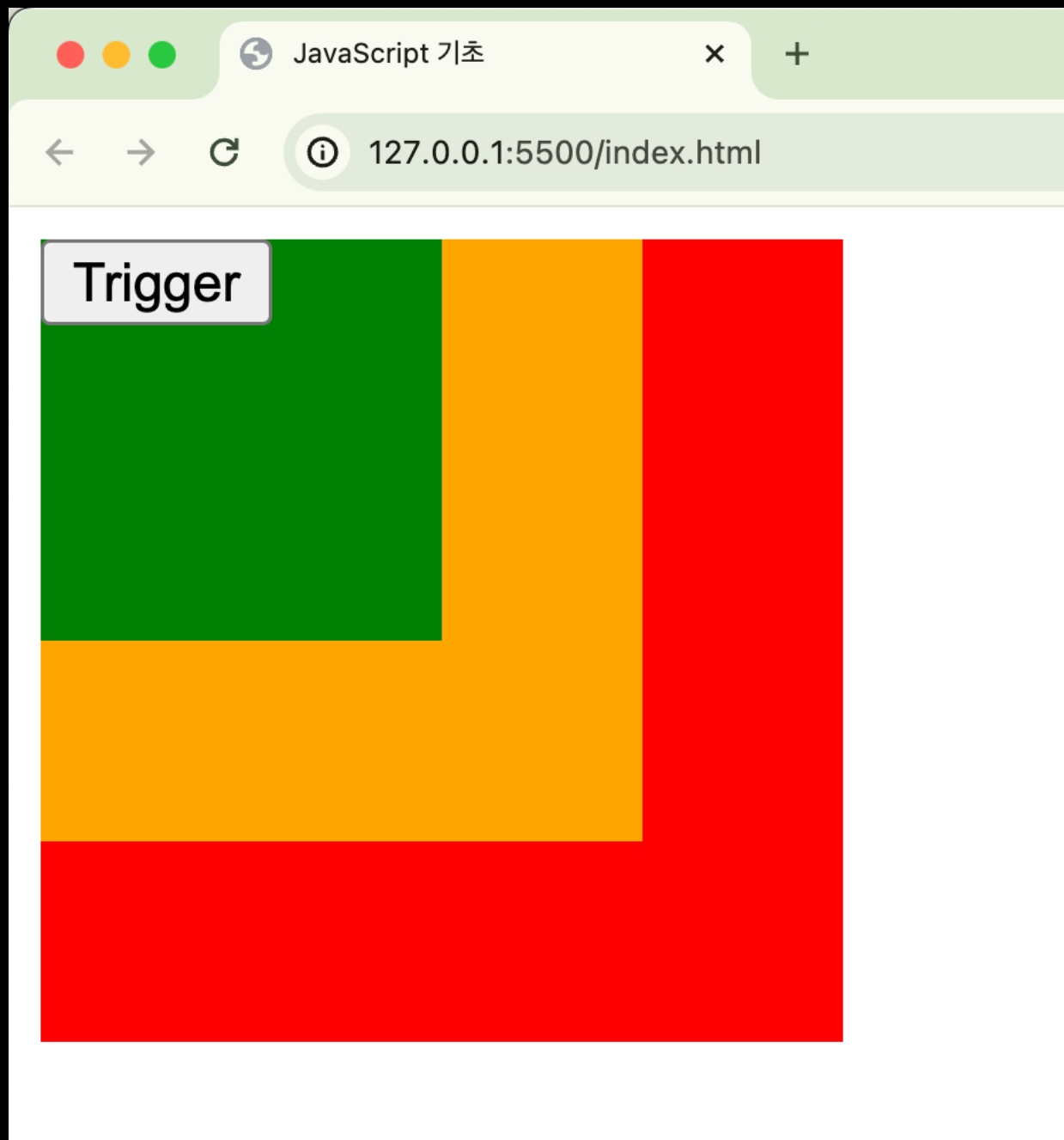


DOM과 이벤트 II



이벤트 propagation



```
<div
  id="parent-1"
  style="width: 200px; height: 200px; background-color: red"
>
  <div
    id="parent-2"
    style="width: 150px; height: 150px; background-color: orange"
  >
    <div
      id="parent-3"
      style="width: 100px; height: 100px; background-color: green"
    >
      <button id="btn">Trigger</button>
    </div>
  </div>
</div>
```

window

document

<html>

<body>

div#parent1

div#parent2

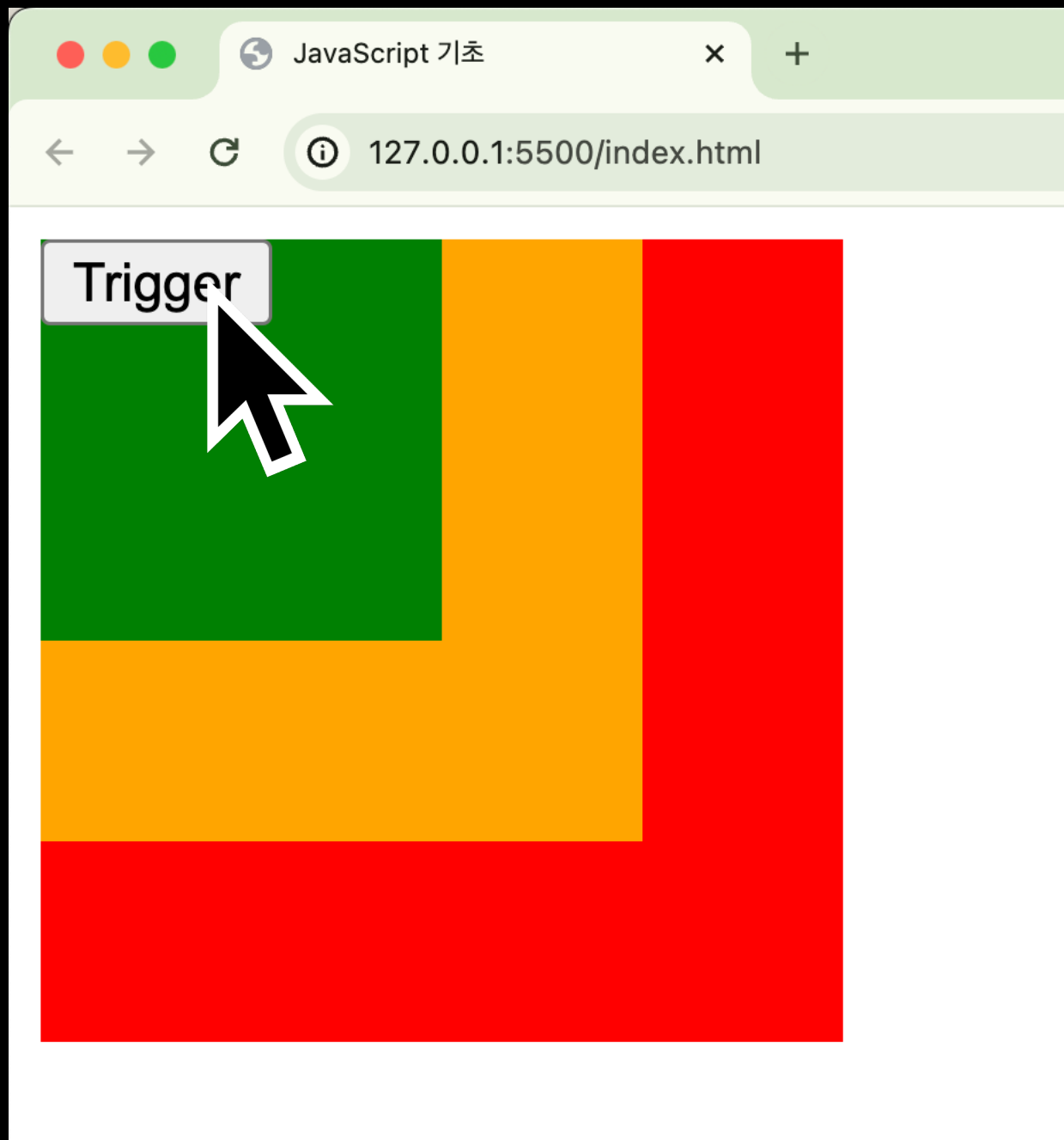
div#parent3

button#btn

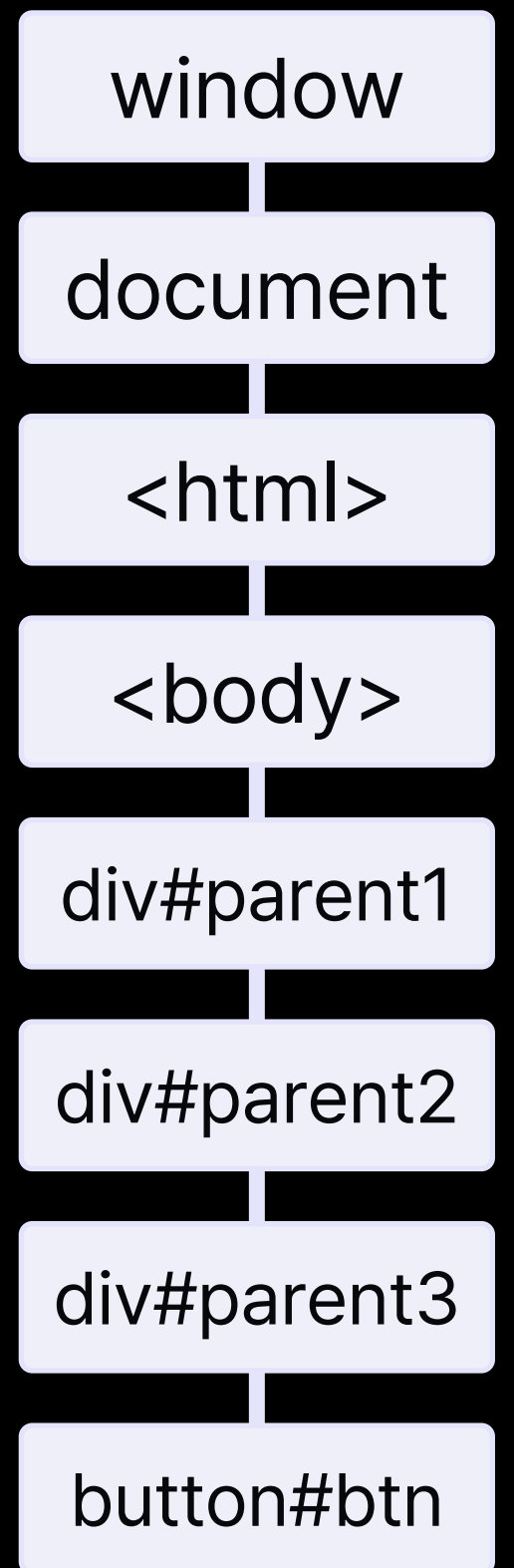
DOM에서 버튼 요소는 3개의 부모 요소와
그 상위 노드인 <body>, <html>, document, window 등이 있다.



이벤트 propagation



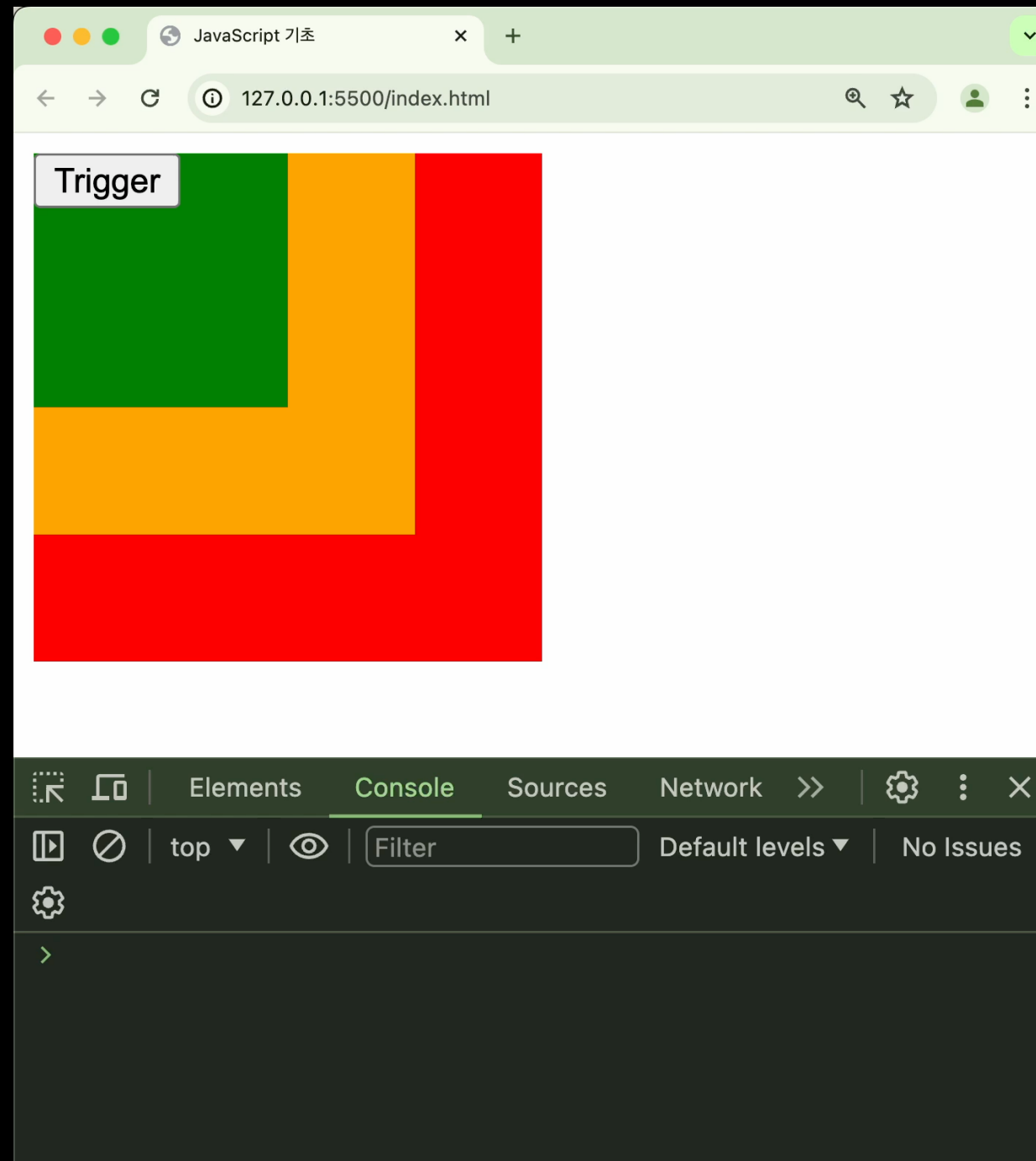
```
btn.addEventListener("click", function (event) { ... });  
parent3.addEventListener("click", function (event) { ... });  
parent2.addEventListener("click", function (event) { ... });  
parent1.addEventListener("click", function (event) { ... });  
...
```



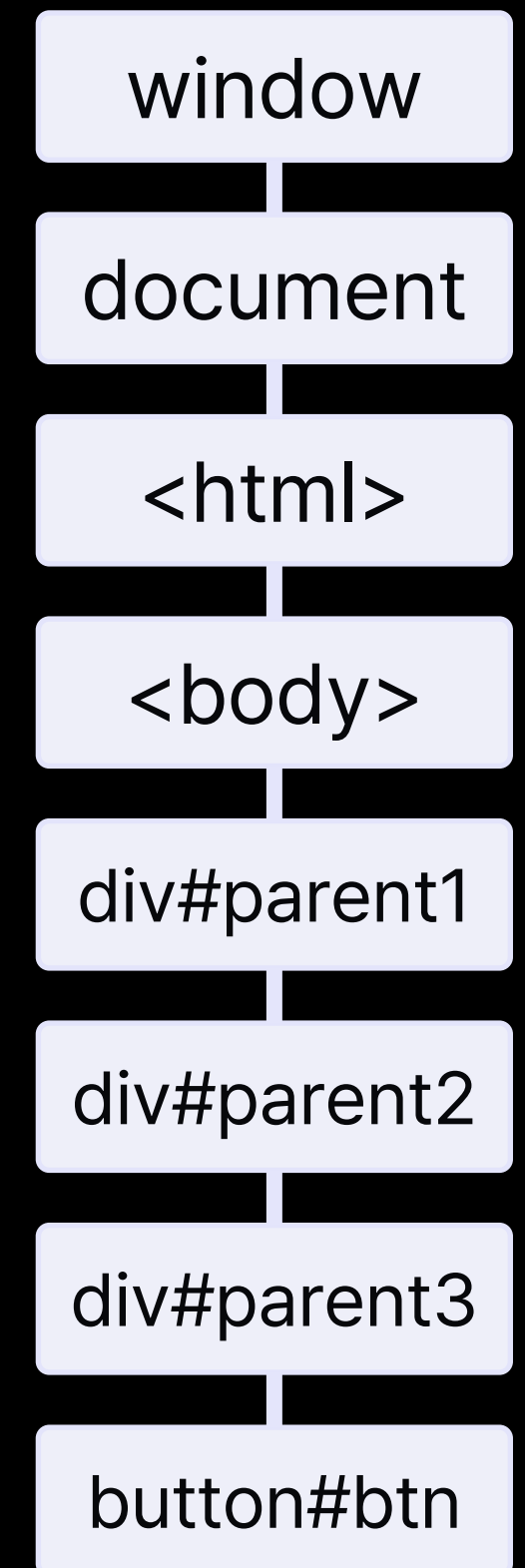
가장 하위 요소인 버튼을 클릭하면, 이 이벤트는 어느 요소에서 발생하는 것일까?



이벤트 propagation



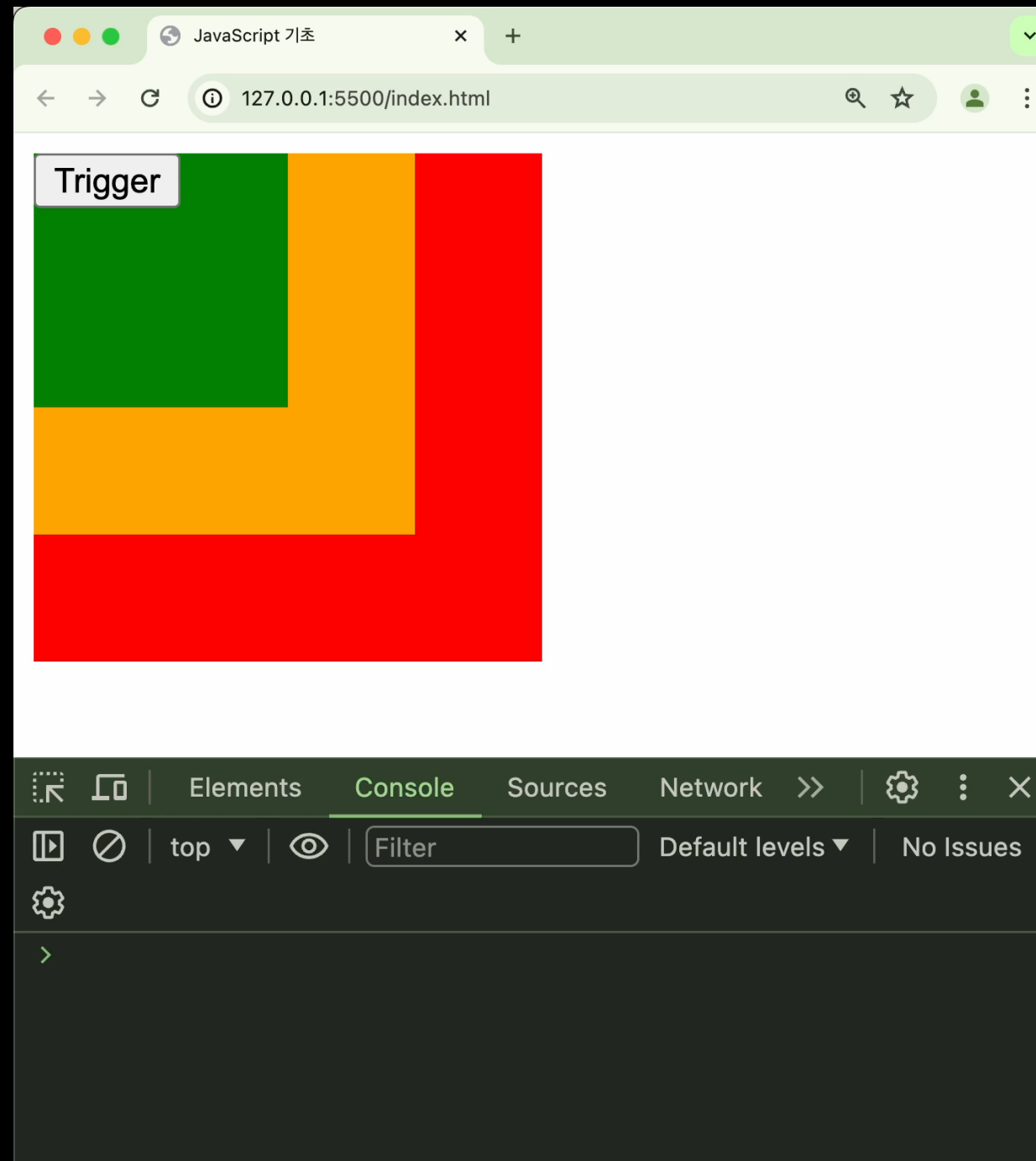
Play →



정답은 **모든 요소**에서 이벤트가 발생한다!
= 모든 요소에서 **이벤트를 캐치**할 수 있다.



이벤트 propagation



Play →

이벤트가 발생하면, 이 이벤트는 **DOM 트리를 통해 전파**된다.

이를 **이벤트 propagation**이라고 한다.

window

document

<html>

<body>

div#parent1

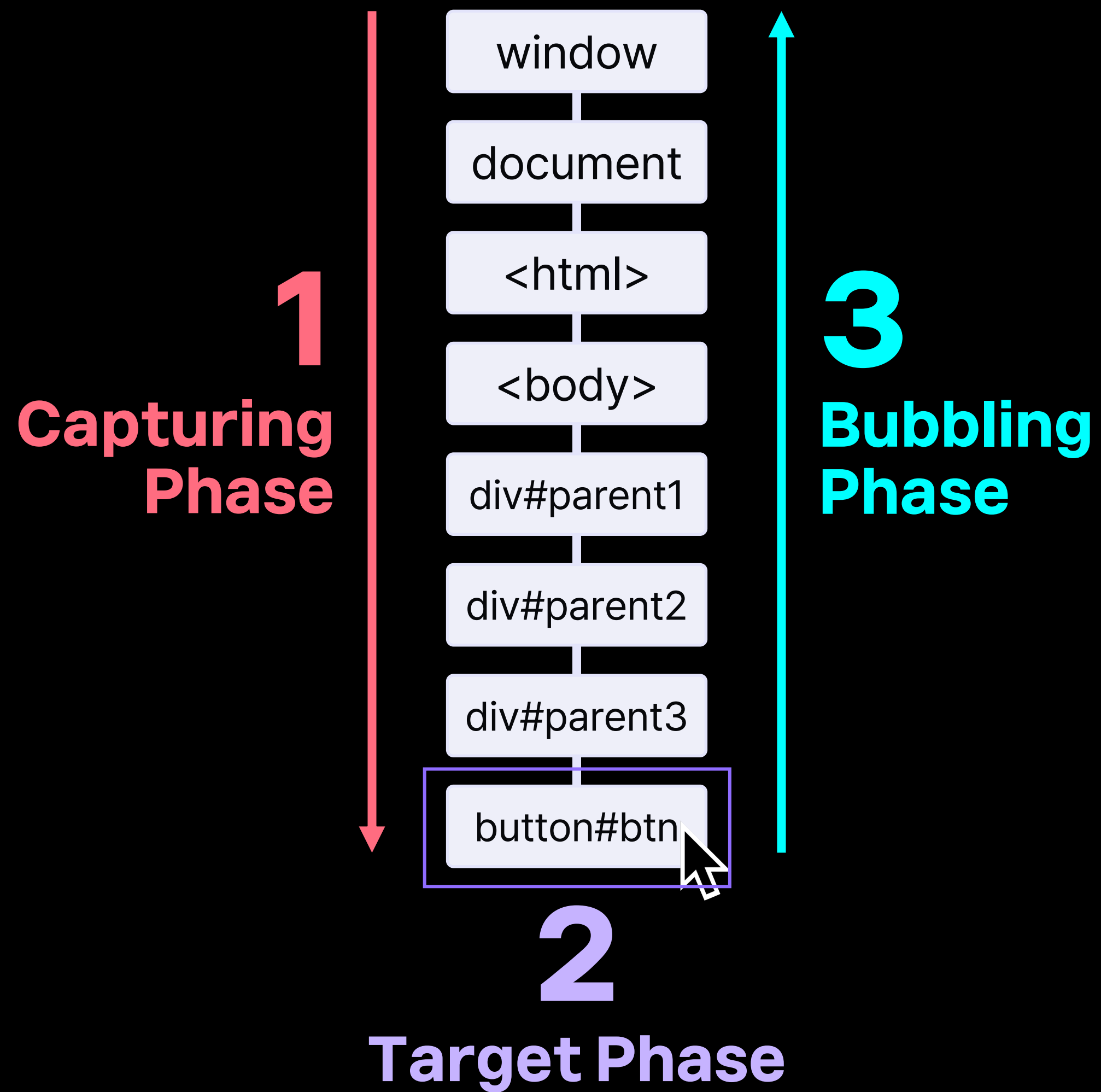
div#parent2

div#parent3

button#btn

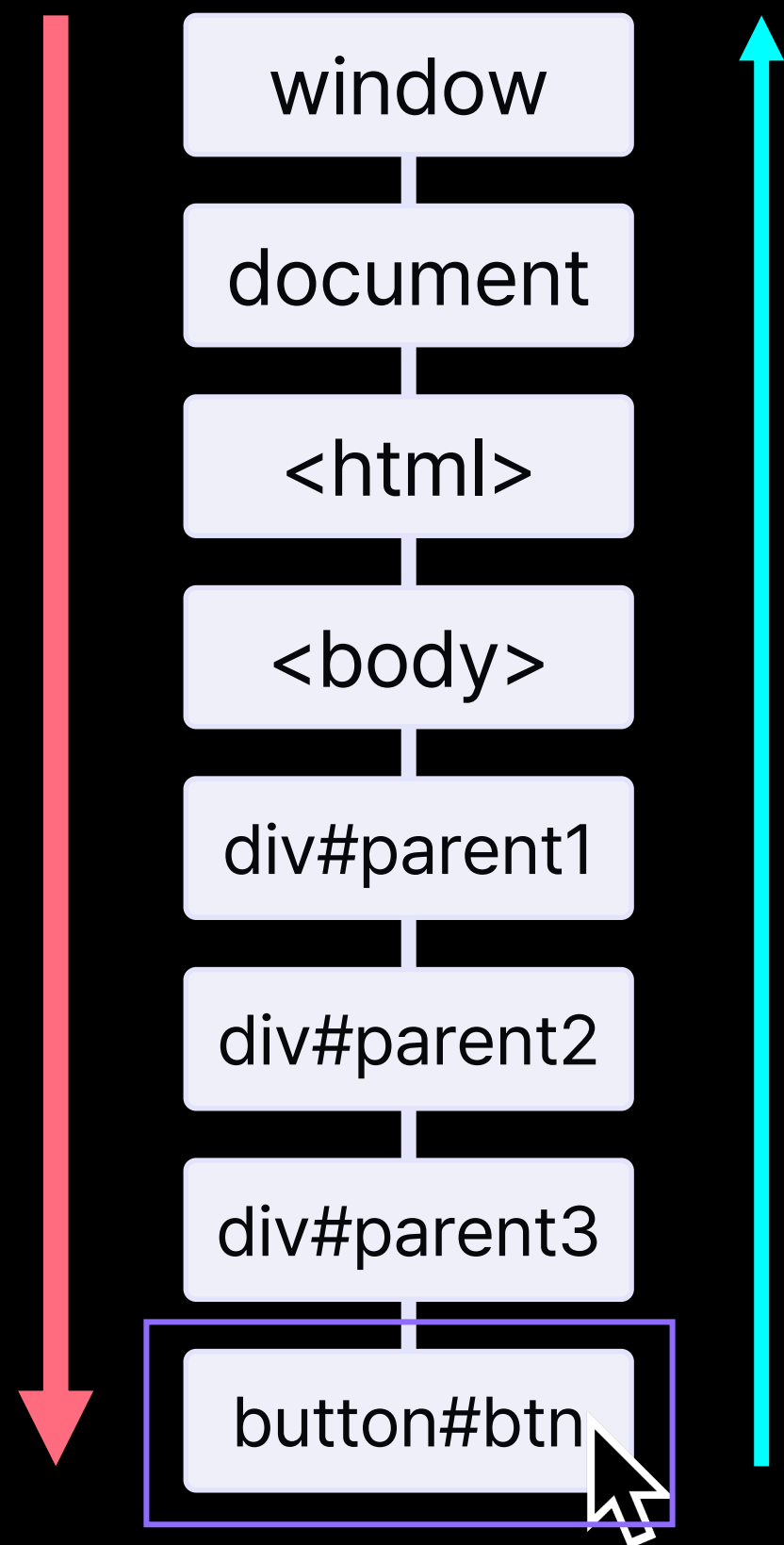


이벤트 propagation





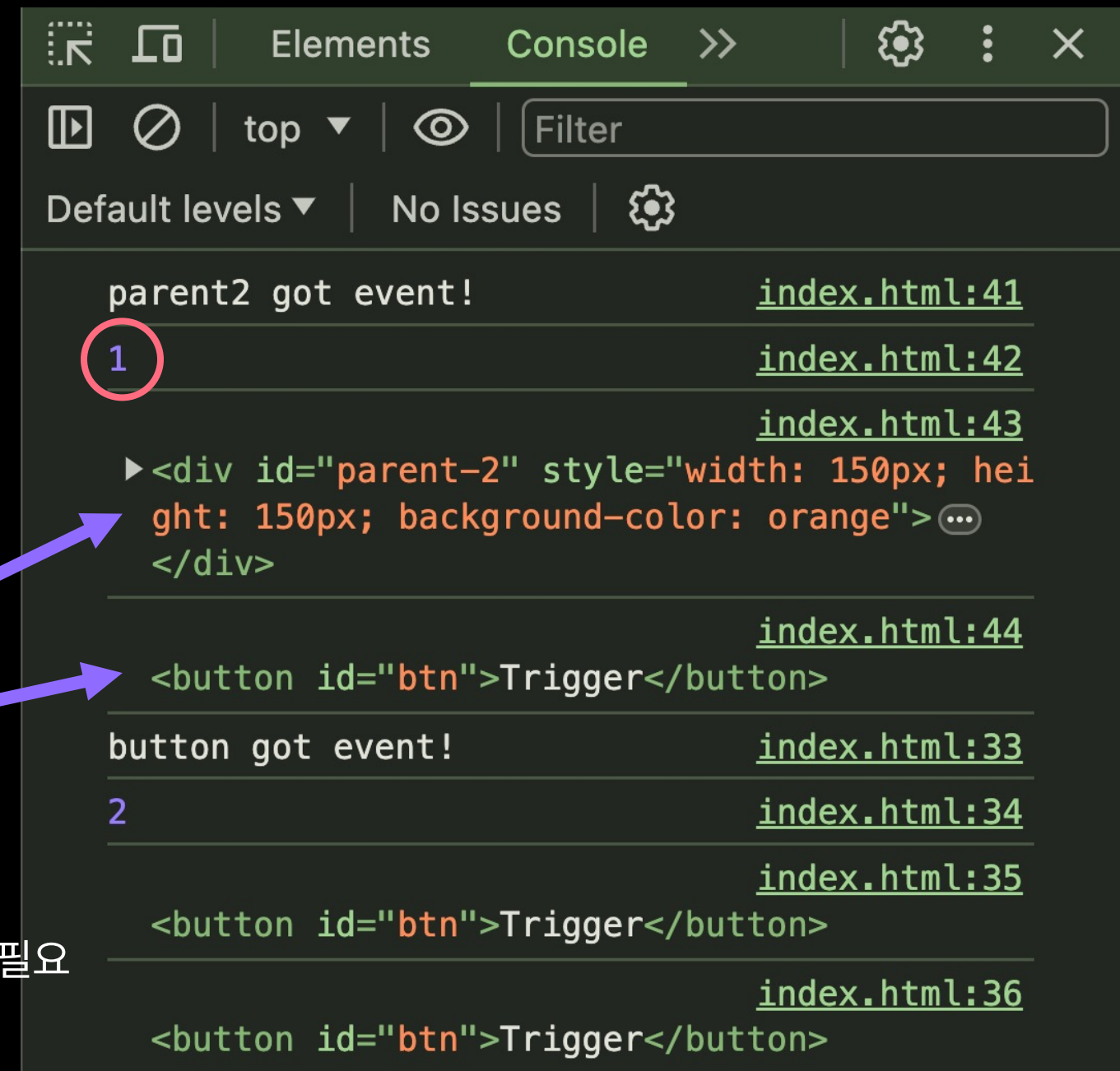
이벤트 propagation



1 Capturing Phase

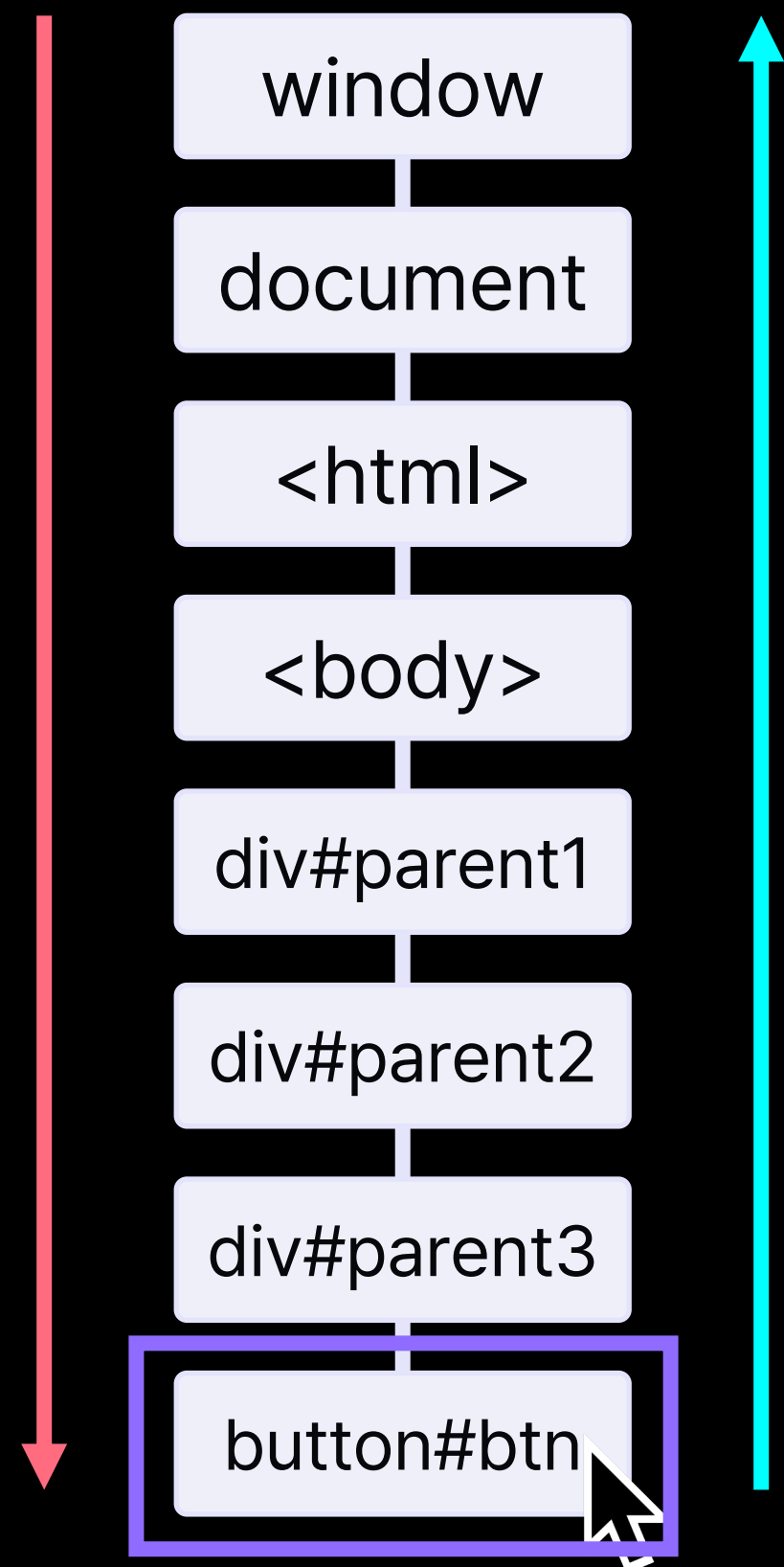
```
btn.addEventListener("click", function (event) {
  console.log("button got event!");
  console.log(event.eventPhase);
  console.log(event.currentTarget);
  console.log(event.target);
});
parent2.addEventListener(
  "click",
  function (event) {
    console.log("parent2 got event!");
    console.log(event.eventPhase);
    console.log(event.currentTarget);
    console.log(event.target);
  },
  true
);
```

Capturing phase에서 이벤트를 캐치하려면
addEventListener의 세 번째 인자로 'true' 전달 필요





이벤트 propagation



2 Target Phase

```
btn.addEventListener("click", function (event) {
  console.log("button got event!");
  console.log(event.eventPhase);
  console.log(event.currentTarget);
  console.log(event.target);
});
parent2.addEventListener(
  "click",
  function (event) {
    console.log("parent2 got event!");
    console.log(event.eventPhase);
    console.log(event.currentTarget);
    console.log(event.target);
  },
  true
);
```

The screenshot shows the browser's developer console with the 'Console' tab selected. It displays a series of log messages corresponding to the event propagation. The messages are as follows:

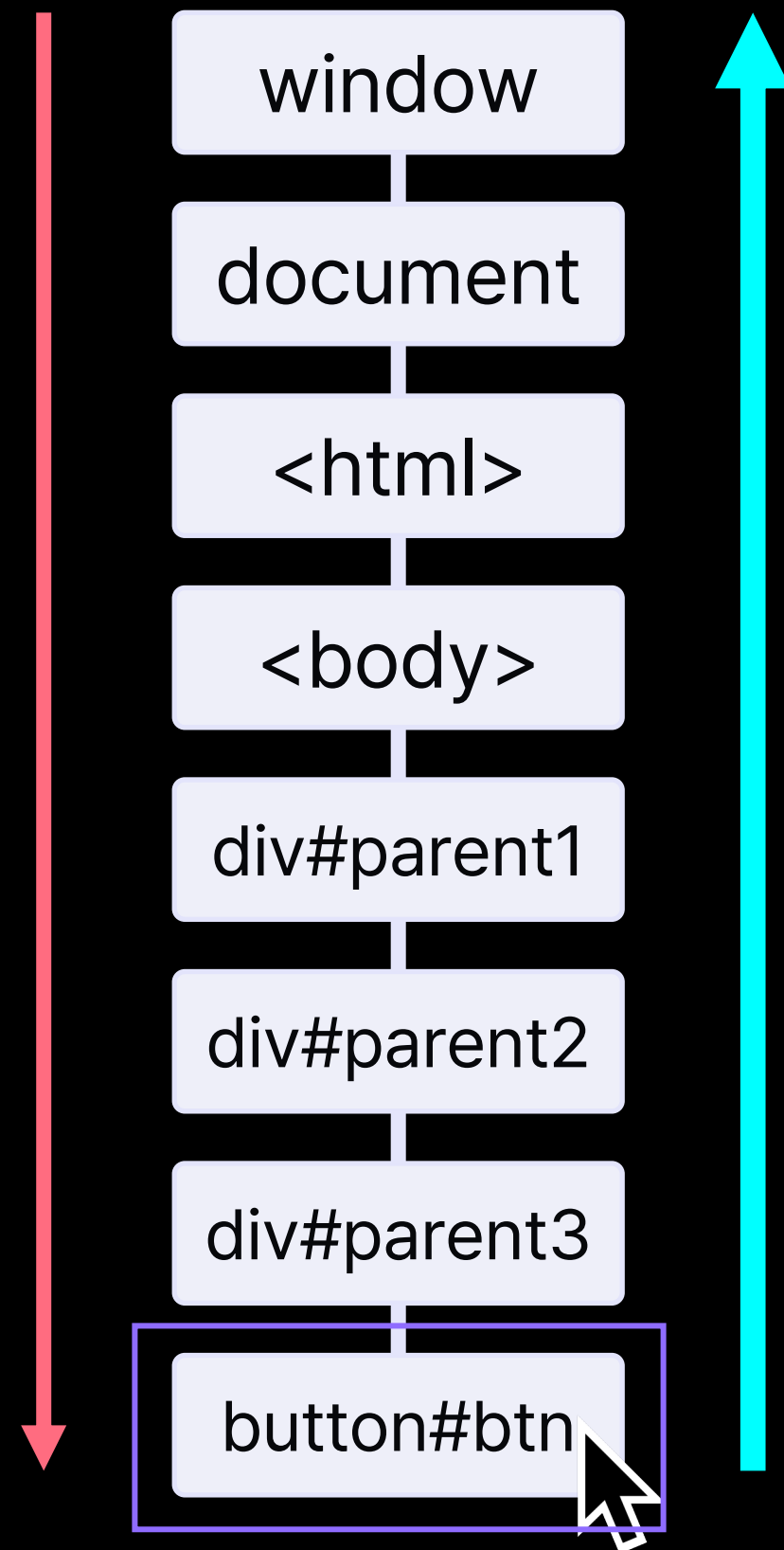
- parent2 got event! (index.html:41)
- 1 (index.html:42)
- index.html:43
- ><div id="parent-2" style="width: 150px; height: 150px; background-color: orange">... (index.html:44)
- <button id="btn">Trigger</button> (index.html:44)
- button got event! (index.html:33)
- 2 (index.html:34)
- index.html:35
- <button id="btn">Trigger</button> (index.html:36)
- <button id="btn">Trigger</button> (index.html:36)

Two blue arrows originate from the code in the previous block. One arrow points from `event.currentTarget` to the log entry `<button id="btn">Trigger</button>` (index.html:35). The other arrow points from `event.target` to the log entry `<button id="btn">Trigger</button>` (index.html:36). The number '2' in the log entry (index.html:34) is circled in red.



이벤트 propagation

3 Bubbling Phase



```
btn.addEventListener("click", function (event) {
  console.log("button got event!");
  console.log(event.eventPhase);
  console.log(event.currentTarget);
  console.log(event.target);
});
parent2.addEventListener("click", function (event) {
  console.log("parent2 got event!");
  console.log(event.eventPhase);
  console.log(event.currentTarget);
  console.log(event.target);
});
```

addEventListener의 세 번째 인자를 전달하지 않으면
기본으로 Bubbling phase에서 이벤트를 캐치

